



金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器 之前端技术架构设计实操

1. 课程代码

 es6-vue-template.zip
30.34KB

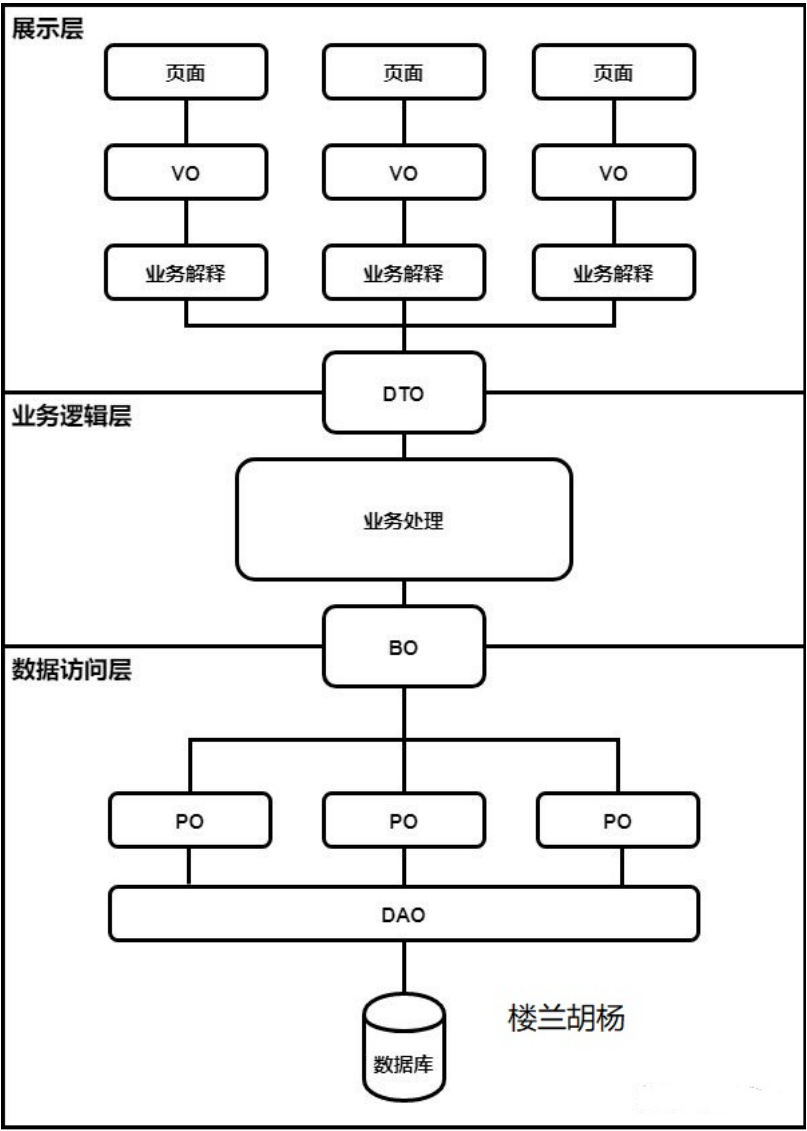
 encode-vue-mobile-template.zip
126.84KB

 vue-template.zip
63.23KB

2. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 -- Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)

3. 项目架构设计



3.1 Vue PC 端模板

- [Vue Router](#)
- [vite-plugin-pages](#): 以文件系统为基础的路由
- [vite-plugin-vue-layouts](#): 页面布局系统
- [Pinia](#): 直接的, 类型安全的, 使用 Composition API 的轻便灵活的 Vue 状态管理
- [vite-plugin-vue-markdown](#): Markdown 作为组件, 也可以让组件在 Markdown 中使用
- [markdown-it-prism](#): [Prism](#) 的语法高亮
- [prism-theme-vars](#): 利用 CSS 变量自定义 Prism.js 的主题
- [Vue I18n](#): 国际化
- [VueUse](#): 实用的 Composition API 工具合集
- [@vueuse/head](#): 响应式地操作文档头信息
- [vite-plugin-vue-devtools](#): 旨在增强 Vue 开发者体验的 Vite 插件
- 使用 Composition API 地 `<script setup>` SFC 语法
- [TypeScript](#)
- [Vitest](#): 基于 Vite 的单元测试框架
- [Cypress](#): E2E 测试

3.2 Vue Mobile 端模板

- [Vue Router](#)
- [vite-plugin-vue-layouts](#): 页面布局系统
- [Pinia](#): 直接的, 类型安全的, 使用 Composition API 的轻便灵活的 Vue 状态管理
- [vant4](#): 轻量、可定制的移动端 Vue 组件库
- [@vant/use](#): 实用的基于 Vant 的 Composition API 工具合集

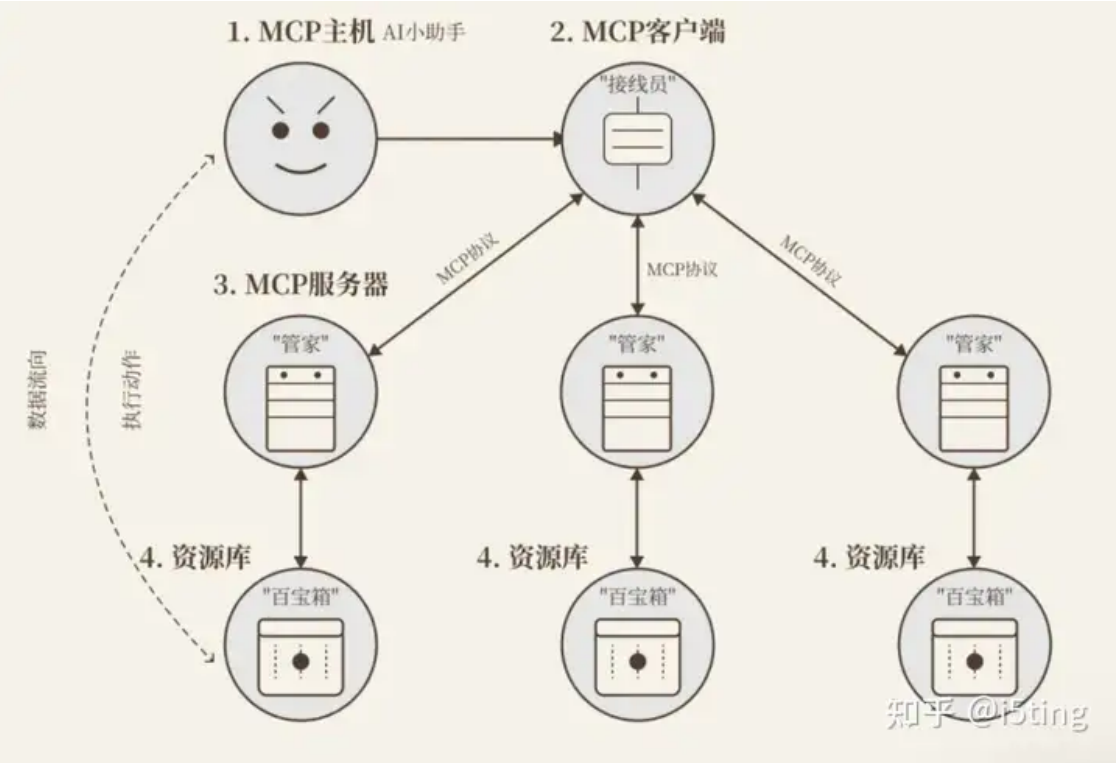
- 使用 Composition API 地 `<script setup>` SFC 语法
- TypeScript
- Vitest：基于 Vite 的单元测试框架

4. 如何回答未来 5 年的职业目标是什么？

想清楚自己是什么样的人。

4.1 前端转型做agent应用开发是主流

前端擅长做应用和交互，最合适的工作的就是做ai agent应用，这个事儿一定会火起来。目前ai基建能卷的，都不是小公司了。但做agent应用，才开始。比如smithery 这个mcp源上才1000多个mcp服务。cline的mcp市场也刚刚开始。mcp解决了api式服务的规范问题。（mcp现在特别粗糙，都是计划）



lightpanda在ai无头浏览器性能也是chrome的11倍，兼容playwrite，Puppeteer（cdp模式）等。通过无头浏览器做自动已经具备可行性。

所以开发agent应用是主流。

- 基建可用。会出新入口，比如smithery，会改变交互，甚至是颠覆。技术难度会比较高
- 业务可用。技术难度低。只要有想法，就可以像发npm包一样提供agent服务，按需收费。

4.2 技术部分探索

对于技术栈，摘自我之前的回答，还比较浅，入门够了。

任何时代 和用户交互 都是避免不了的，交互变革也会随着agent变多和管理而改变，又是一波新机会。这里通过5步开源项目帮大家把技术串联起来。

1. ai入门：ai-chatbot

为了学习，和探索，可以研究了一下<https://github.com/vercel/ai-chatbot>，这是一个非常好的易于学习的Node.js AI Agent开发项目模版，各种最新的技术栈基本都覆盖了。

- a. 基于next，ai-sdk，实现基于openai模型的chatbot功能，技术栈和功能很实用。
- b. 足够小，100个ts文件，代码行数10000行左右，结构清晰，简单易学。
- c. 前后端，典型的全栈应用，包含postgre db，鉴权，react，ai，非常全面了。很多最佳实践，比如toast库，密码加盐处理等。
- d. 可以使用aihubmix等openai代理服务，支持支付宝。
- e. 扩展性强，使用其他模型，以及langchain、llamaindex等。

部署还是有点麻烦，涉及的内容还是比较多的。

学完这个，你发现很多网站，网页，都是这个上面缝缝补补的。

2. ai搜索：scira

这个开源项目也很简单，非常适合入门。如果已经熟悉了ai-sdk，这里再学一些搜索服务集成（比如Tavily AI），也是极好的。它在应用层面上，是非常简单且实用的示例。

3. 深挖基础lib：langchain 和 llamaindex

深挖langchain和llamaindex，没啥好说的，这是ai世界里，构建rag必备的包，都是现有py版本，然后才有js/ts版本，可见在应用开发领域，js/ts依然是最受欢迎的方式。

4. 探索客户端，基于vite + electron-forge，另外还有很多rust实现

<https://github.com/block/goose>

其中agent、扩展，以及类似于Eventloop的机制，有很多很有意思的实现。

5. 探索应用开发。上面的技术基本上都包含了，比如llamaindex

<https://github.com/mastra-ai/mastra>

基本上把各种ai sdk都集成了，agent、tool、workflow、rag等全部支持，开箱即用，写起来更简单。

5. 具体技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



6. 前端项目经验相关面试考察点

1. 所谓的面试，是在一众简历中选择最合适的候选人。面试之所以卷，不是指简历的内容写的多或者项目多就好，而是在跟其他人竞争时，你有别人的简历中所没有的亮点，能够体现出你的技术水平，这才算好。所谓的行情不好，只是原先从简历中选择 top 50%降低到了top 25%，但有亮点的简历，始终是在top 20%以内的，所以没有亮点和深度的简历之间越来越卷，越来越供大于求；但有亮点有技术深度的人始终是处于供不应求的情况；
2. 面试时，技术水平的高低始终是第一位的，如果水平足够，但学历低，优先级也会比学历高但水平低的人高；当然，如果水平一样，肯定有限学历高的候选人；
3. 现在的面试都是通过项目切入，根据项目中具体的实现的功能来引导使用的技术栈，考察的是针对对应的技术栈，具体掌握的深度和广度；
4. 面试的时间一般不会超过1小时，在1小时里不可能把所有的前端内容全部问完，所以面试官会挖掘候选人的项目亮点，通过候选人的面试，考察候选人对知识技能的掌握，再逐步提升难度，摸透候选人对知识掌握的边界；
5. 面试的过程是发掘出候选人优势的过程，不是打压候选人的过程，所以不用担心个别地方说的不好会不会导致面试不过，如果亮点项目很充分，能够体现出个人水平，面试官也不会在意一些边支末节的。

项目经验通常和个人经历关系比较大，前端业务相关的的一些项目经验通常包括管理端、H5 直出、Node.js、可视化，另外还包括参与工具开发的经验，方案选型、架构设计等。

项目相关的内容，比如性能优化、前端框架之类的

6.1 前端框架与工具库

首先我们来看看前端框架，不管你开发管理端、PC Web、H5，还是现在比较流行的小程序，总会面临要使用某一个框架来开发。因此，以下的问题可能与你有关：

- 谈谈你对前端常见的框架（Angular/React/Vue）的理解
- 该项目使用 Angular/React/Vue 的原因是
- 如果现在你重新决策，你会使用什么框架
- 你有了解过这些框架都做了哪些事情，介绍一下是怎么实现的
- Vue 中的双向绑定是怎么实现的？
- 讲讲 React 的虚拟 DOM
- 如何进行状态管理，Vuex/Redux/Mobx 等工具是怎么做的
- 单页应用是什么？路由是如何实现的
- 如何进行 SEO 优化

如果你使用到了小程序，还可能会问到：

- 小程序和 H5 有什么不一样，为什么选小程序而不是 H5
- 有考虑在小程序里嵌 H5 实现吗，为什么
- 为什么小程序的性能要好一些
- 小程序开发有用到哪些框架吗、为什么

而工具库相关的就太多了，一般会这么问：

- 有实际使用过哪些第三方库
- 这些工具库有什么特性和优缺点

项目相关的许多问题，其实是我们工作中经常会遇到并需要进行思考的问题。如果平时有养成思考和总结的习惯，那么这些问题很容易就能回答出来。如果平时工作中比较少进行这样的思考，也可以在面试准备的时候多关注下。

6.2 Node.js 与服务端

Node.js 相关的可能包括：

- 为什么要用 Node.js（而不是 PHP/JAVA/GO/C++等）

- Node.js 有哪些特点，单线程的优势和缺点是什么
- Node.js 有哪些定时功能
- `Process.nextTick` 和 `setImmediate` 的区别
- Node.js 中的异步和同步怎么理解，异步流程如何控制
- 简单介绍一下 Node.js 中的核心内置类库(事件，流，文件，网络等)
- express 是如何从一个中间件执行到下一个中间件的
- express、koa、egg 之间的区别
- Rest API 有使用过吗，介绍一下

以上这些都属于很基础的问题。很多时候，我们会使用 Node.js 去做一些脚本工程或是服务端接入层等工作。如果项目中有使用 Node.js，面试官更多会结合项目相关的进行提问。

6.3 性能优化

性能优化的其实跟项目比较相关，常见的包括：

- 有做过性能优化相关的项目吗，具体的优化过程是怎样的/优化效果是怎样的
- 常见的性能优化包括哪些内容
- 如何理解项目的性能瓶颈/什么时候我们需要对一个项目进行优化
- 图片加载性能有哪些可以优化的地方
- 要怎么做好代码分割/降低代码包大小可以有哪些方式
- 首屏页面加载很慢，要怎么优化
- Tree Shaking 是怎样一个过程
- 页面有没有做什么柔性降级的处理

很多时候，性能优化也是与项目本身紧紧相关，一般来说会包括首屏耗时优化、页面内容渲染耗时优化、内存优化等，可能涉及代码包大小、下载耗时、首屏直出、存储资源（内存/indexDB）等内容。

6.4 前端工程化

如今前端工程化的趋势越来越重，通常从脚手架开始：

- 为什么我们开发的时候要使用脚手架
- 如何理解模块化
- 为什么要使用 Webpack，它和 Gulp 的区别是
- 讲一下 Webpack 中常用的一些配置、Loader、插件
- Babel 的作用是什么，如何选择合适的 Babel 版本
- Webpack 是怎么将多个文件打包成一个，依赖问题如何解决
- 有写过 Webpack 插件吗，Webpack 编译的过程具体是怎样的
- CSS 文件打包过程中，如何避免 CSS 全局污染
- 本地开发和代码打包的流程分别是怎样的

除了脚手架相关，如今自动化、流程化的使用也越来越多了：

- 怎么理解持续集成和持续部署
- 你们的项目有使用 CI/CD 吗，为什么
- 你们的代码有写单元测试/自动化测试吗，为什么
- 前端代码支持自动化发布吗，如何做到的

工程化和自动化是如今前端的一个趋势，由于团队协作越来越多，如何提升团队协作的效率也是一个可具备的技能。

6.5 开发效率提升

效能提升的意识在工作中很重要，大家都不喜欢低效的加班。通常可能问到的问题包括：

- 做了很多的管理端/H5，有考虑过怎么提升开发效率吗
- 你的项目里，有没有哪些工作是可以的工具完成的
- 项目中有进行组件和公共库的封装吗
- 如何管理这些公共组件/工具的兼容问题
- 日常工作中，如何提升自己的工作效率

6.6 监控、灰度与发布

发布和监控这部分，可能较大的业务才会有，涉及的问题可以有：

- 日常开发过程中，怎么保证页面质量
- 版本发布有进行灰度吗？灰度的过程是怎样的
- 版本发布过程中，如何及时地发现问题
- 发生异常，要怎么快速地定位到具体位置
- 如何观察线上代码的运行质量

对于大型项目来说，灰度发布几乎是开发必备，而监控和问题定位也需要各式各样的工具来辅助优化。

6.7 多人协作

一些较大的项目，通常由多个开发合作完成。而多人协作的经验也很有帮助：

- 多人开发过程中，代码冲突如何解决
- 项目中有使用 Git 吗？介绍一下 Git flow 流程
- 如果项目频繁交接，如何提升开发效率
- 有遇到代码习惯差异的问题吗，如何解决
- 有哪些常用的代码校验的工具
- 怎么强制进行 Code Review

看到这么多内容不要慌，一般来说面试官只会根据你的工作经历来询问对应的问题，所以如果你并没有完全掌握某一块的内容，请不要写在简历上，你永远也不知道面试官会延伸到哪

7. 师资&课程内容

1. 师资：一线大厂/外企现役P8级高级专家亲自带；
2. 内容：深度对标阿里P6-P7，涵盖大厂所有内训标准；

7.1 项目实战内容

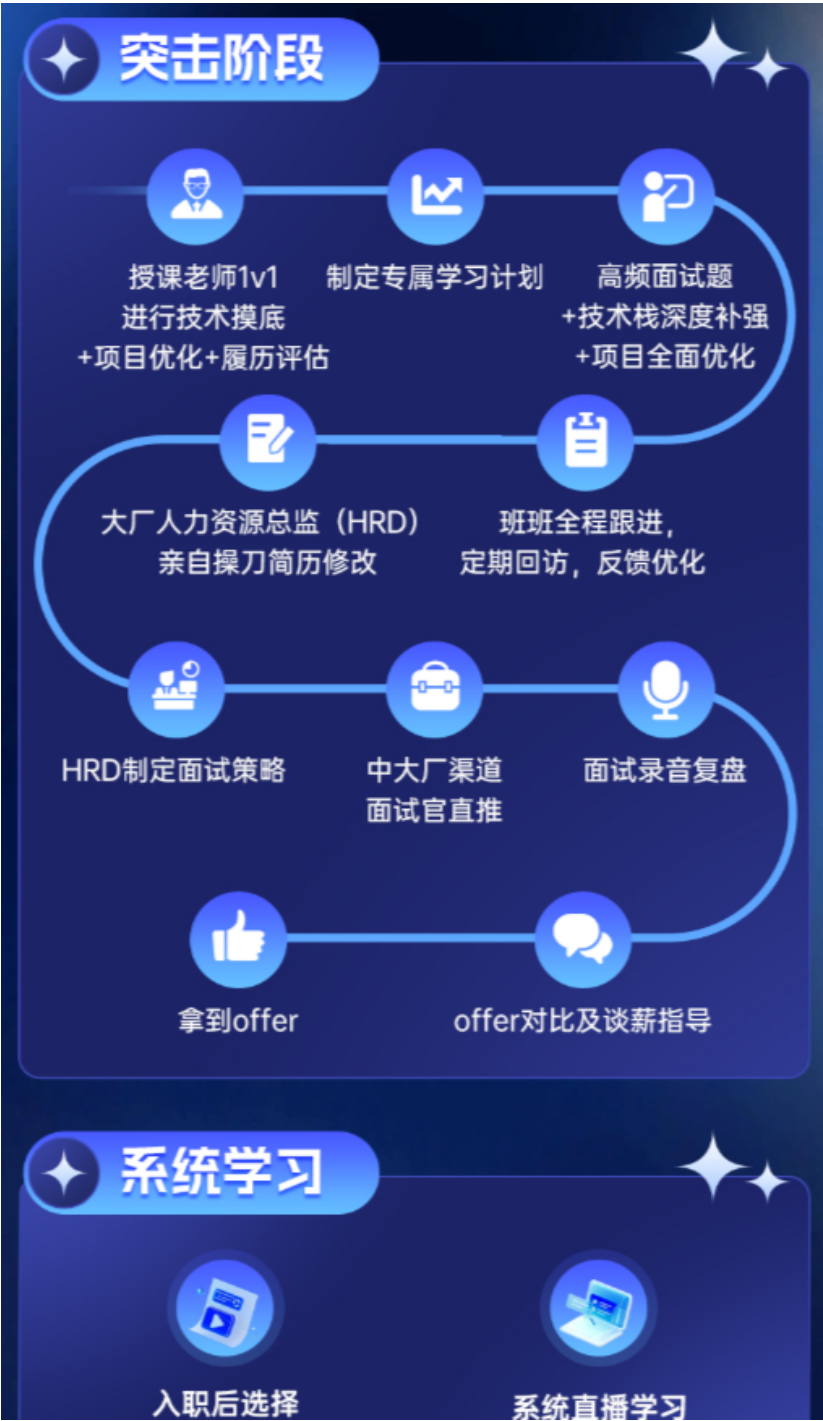
github：<https://github.com/encode-studio-fe>

[目前端实战课分享汇总](#)（可咨询班主任要密码权限）

7.2 服务内容

系统&突击两手抓

全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



7.3 专业一对一

近期一对一

- 📅 (2025-02-06) xxx-2.5年-React-24K
- 📅 (2025-02-07) xxx-1年-React-15k
- 📅 (2025-01-05) xxx-7年-Vue-20k

报名学员专享

📖 13. 如何写出高质量的简历 (For 付费学员)

7.4 渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

7.5 近期报名学员案例

